HAC: Node-Level Hybrid Force–Graph Control and Smooth Handover Scheduling for Resilient UAV Swarm Connectivity Recovery

Abstract

Self-healing communication connectivity (SCC) for unmanned aerial vehicle (UAV) swarms requires rapidly reconnecting fractured subnetworks after partial node failures while preserving wide spatial coverage, yet pursuing either objective in isolation directly degrades the other. Recent hybrid frameworks such as R2C address this tension by combining intra-subnetwork virtual-force expansion with subnetwork-level multipartite graph convolution under a steep time-adaptive fusion schedule, bundling together several unisolated architectural choices. In this work, we formulate a three-axis taxonomy of the SCC design space spanning decision paradigm, spatial reasoning granularity, and trade-off scheduling, and systematically isolate each axis through five-seed controlled experiments. We propose Hybrid Adaptive Convolution (HAC), a lightweight node-level hybrid controller that fuses a parameter-free buffered virtual-force heuristic with a single global graph convolutional policy at the velocity level without subnetwork partitioning, convex-hull boundary extraction, or multipartite graph construction. Across random and uniform deployment scenarios under up to 80% node failure, HAC under a constant 50/50 fusion weight (HAC-Fixed) outperforms purely heuristic, purely learned, and subnetwork-level rigid-translation baselines with substantially lower cross-seed variance and 1.5× to 2.2× faster execution, while retaining 88.4% to 91.2% of the coverage reported by R2C (5.9 to 7.9 pp absolute difference) under 80% node loss. Furthermore, we show that transplanting steep adaptive fusion schedules into simplified hybrid controllers severely degrades recovery robustness, whereas widening the transition window restores high recovery performance. By conducting a 3×3 decoupled training-versus-evaluation schedule study, we falsify the hypothesis that steep schedules corrupt policy learning during training and prove instead that premature inference-time handover extinguishes the heuristic link-retention barrier during long-horizon rollouts. Finally, granularity-substitution counter-experiments confirm that smooth scheduling rescues node-level policies but cannot rescue subnetwork-level rigid controllers that lack node-level actuation degrees of freedom.

1 Introduction

1.1 Problem Context and Fundamental Trade-Off

Unmanned aerial vehicle (UAV) swarms deployed in contested or disaster-struck environments face recurring partial node failures that fracture communication topologies and degrade spatial coverage. Wireless aerial networks provide rapid communication coverage in post-disaster relief, dynamic battlefields, and congested areas (Coletta et al., 2023; Shi et al., 2025; Wang et al., 2022, 2024b). During operation, internal hardware faults or external attacks can disable a subset of UAVs at any time (Wang et al., 2024a; Kurunathan et al., 2023). When substantial node loss occurs, the surviving communication graph splits into multiple disconnected components, severely disrupting both multi-hop data routing and mission-area coverage (Li et al., 2023; Peng

et al., 2022). The self-healing communication connectivity (SCC) problem seeks a velocity control policy that repositions surviving UAVs so that the communication graph regains full connectivity as rapidly as possible while preserving maximal spatial coverage across the mission area.

Accelerating topological reconnection and preserving spatial coverage impose directly opposing kinematic demands on surviving agents. Pulling surviving UAVs inward toward a common center shortens reconnection time but contracts the covered footprint, whereas dispersing UAVs outward preserves area coverage at the risk of slowing or preventing link restoration. Managing this tension becomes increasingly difficult as the failure ratio approaches extreme regimes where surviving nodes are sparsely scattered across the arena.

1.2 Limitations of Existing Paradigms

Purely heuristic recovery methods require no training data but scale poorly under severe, unstructured node failures. Classical non-learning approaches either identify critical cut-vertices and trigger cascaded neighbor substitutions (Akkaya et al., 2009; Mi and Yang, 2015; Zhang et al., 2024) or guide surviving agents toward predefined rendezvous points using artificial potential fields (Joshi, 2012; Shriwastav, 2018; Chen et al., 2020; Wang et al., 2024c). Because these methods rely on static topological assumptions and local interaction rules, they become brittle when large-scale failures isolate components beyond local sensing ranges (Tosun et al., 2024).

Purely learned graph neural network (GNN) policies achieve rapid reconnection but suffer from spatial collapse. Learning-based methods formulate SCC as a differentiable graph optimization or reinforcement learning problem (Mou et al., 2022; Peng et al., 2025; Wang et al., 2025b). While GNN policies adapt effectively to diverse failure topologies, neighborhood message aggregation biases surviving UAVs to congregate around high-degree hubs, causing severe spatial clustering and coverage loss. Diffusion-based and differential-subgraph convolutions (Lin et al., 2024; Lin and Ding, 2025) partially alleviate clustering by promoting multi-hop dispersion, yet they struggle to maintain stable layouts under extreme fragmentation and rarely enforce inter-agent spacing guarantees.

1.3 Disentangling Hybrid Recovery Architecture and Scheduling

Recent divide-and-conquer hybrid frameworks achieve strong performance but bundle spatial granularity with steep adaptive scheduling. To reconcile connectivity and coverage, Peng et al. (2026) introduced R2C, which combines a buffered virtual-force module for intra-subnetwork expansion (ISE) with a multipartite graph convolution module for inter-subnetwork translation (IST) governed by an adaptive weighted fusion (AWF) schedule. Because R2C's ablations toggle entire modules on or off, two fundamental design questions remain open: whether the learned branch must operate at subnetwork granularity with explicit convex-hull boundary extraction rather than globally at the node level, and whether adaptive fusion requires a steep temporal transition rather than a smooth or constant weighting.

Controlled multi-seed ablations reveal that node-level hybrid fusion with fixed or smooth inference scheduling captures the vast majority of recovery performance without geometric partitioning. We investigate these questions through Hybrid Adaptive Convolution (HAC), a streamlined architecture that fuses a parameter-free buffered virtual-force branch with a single global graph convolutional policy at the velocity level. Across five independent random seeds per configuration, our study yields four central findings:

1. Fusing a global node-level learned branch with a parameter-free virtual-force heuristic under a constant 50/50 weight (HAC-Fixed) retains 88.4% to 91.2% of R2C's reported coverage

under 80% node failure while achieving near-perfect recovery rates (46.6 ± 3.9 and 49.4 ± 0.8 out of 50 trials) and substantially lower variance than single-branch baselines.
2. Directly porting a steep adaptive fusion schedule ($k = 30, s_0 = 0.125$) to this simplified architecture degrades 80%-failure recovery to 14.2 ± 4.3 out of 50 trials, whereas smoothing the transition slope ($k = 5, s_0 = 0.7$) restores recovery to 43.6 ± 2.8 (random) and 47.8 ± 1.3 (uniform).
3. Decoupling training and evaluation schedules in a 3×3 cross-evaluation matrix falsifies the hypothesis that steep schedules impair policy representation learning during training: policies trained under a steep schedule achieve 46.6 ± 0.5 and 46.4 ± 0.5 successes when evaluated under fixed or smooth schedules, proving that collapse is an inference-time phenomenon caused by prematurely extinguishing the heuristic link-retention barrier.
4. Schedule smoothing rescues node-level hybrid control (HAC) but fails to rescue subnetwork-level rigid-translation hybrid control (HSC, which achieves 0.0 ± 0.0 to 0.4 ± 0.5 successes at 80% failure across all schedules), cleanly separating an inference-time handover failure mode from an architectural expressiveness bottleneck.

1.4 Summary of Contributions

The primary contributions of this paper are fourfold:

1. We formulate a three-axis taxonomy of the SCC design space spanning decision paradigm, spatial reasoning granularity, and trade-off strategy, positioning thirteen representative methods and isolating previously unexplored hybrid configurations.
2. We propose HAC, a lightweight hybrid recovery architecture trained via a cardinality-invariant soft-minimum connectivity surrogate loss, and validate its resilience, computational efficiency, and variance-suppression properties across random and uniform deployments over five random seeds.
3. We establish a theoretical and empirical diagnosis of hybrid fusion scheduling, combining a formal link-retention barrier analysis with a 3×3 decoupled training-evaluation matrix that isolates inference-time handover instability from training-time curriculum effects.
4. We provide a five-seed comparative analysis separating node-level versus subnetwork-level hybrid expressiveness, accompanied by open-source simulation code, training scripts, and complete multi-seed evaluation logs.

The remainder of this paper is organized as follows. Section 2 reviews related work, and Section 3 formalizes the SCC problem and design taxonomy. Section 4 presents motivating experiments on single-branch failure modes. Section 5 details the HAC architecture and cardinality-invariant loss design, followed by theoretical analysis in Section 6. Sections 7 through 9 report the experimental setup, main five-seed benchmarks, and decoupled ablation studies. Section 10 discusses design implications and limitations, and Section 11 concludes the paper.

2 Related Work

2.1 Non-Learning Self-Healing Methods

Critical-node compensation algorithms restore local links through cascaded neighbor movements but incur prohibitive coordination overhead as swarm scale grows. Early self-healing protocols detect cut-vertices whose removal partitions the network and trigger sequential backup replacements (Abbasi and Younis, 2008; Akkaya et al., 2009; Mi and Yang, 2015; Zhang et al., 2024). While effective for isolated single-node faults, identifying critical nodes and coordinating

multi-hop cascades scale poorly when simultaneous failures shatter the topology into numerous disjoint clusters.

Rendezvous-point aggregation methods eliminate cut-vertex search but rely on static geometric rules that falter under unpredictable failure patterns. Alternative heuristic schemes direct surviving UAVs or fractured subnetworks toward predetermined landmarks using Coulomb-inspired forces (Joshi, 2012), round-table negotiation (Shriwastav, 2018), or artificial potential fields (Chen et al., 2020; Wang et al., 2024c). Although parameter-free and computationally lightweight, these hand-crafted rules lack global topological awareness and frequently stall in local equilibria or cause excessive coverage contraction under high failure ratios (Tosun et al., 2024).

2.2 Learning-Based Self-Healing Methods

Graph neural network policies accelerate global reconnection but exhibit severe spatial collapse around high-degree nodes. Posing connectivity recovery as a differentiable graph optimization problem enables end-to-end learning of coordinated trajectories (Karalias and Loukas, 2020; Peng et al., 2025). Mou et al. (2022) employ a relaxed GCN over virtual communication links, while Guo et al. (2024) incorporate spatiotemporal multi-head attention. Despite fast convergence, standard message-passing schemes pull peripheral agents toward dense clusters, degrading coverage uniformity.

Diffusion and differential-subgraph architectures improve spatial dispersion yet remain vulnerable to extreme topological fragmentation. Recent works counter spatial collapse by introducing multi-hop diffusion convolutions (Lin et al., 2024) or bipartite message passing over differential subgraphs (Lin and Ding, 2025). While these mechanisms encourage broader spatial spread, purely learned policies lack hard repulsive buffers to guarantee inter-agent spacing and exhibit high sensitivity to weight initialization when up to 80% of the swarm is lost.

2.3 Hybrid Heuristic–Learning Architectures and Positioning

Hybrid recovery frameworks combine physical priors with learned graph representations to balance local spacing and global bridging. Peng et al. (2026) proposed R2C, which pairs a buffered Hookean force field for intra-subnetwork expansion (ISE) with a multipartite spectral GCN for inter-subnetwork translation (IST) using Quickhull boundary extraction (Barber et al., 1996) and asymmetric spectral filtering (Liu and Wang, 2025), blended via a sigmoid adaptive weighted fusion (AWF) schedule (Elfwing et al., 2018). Elsewhere, Li et al. (2025) combine distributed multi-agent reinforcement learning with artificial potential fields at the node level under fixed weighting, while Wang et al. (2025b) and related multi-robot coordination studies (Ye et al., 2022; Mutzari et al., 2025; Shi et al., 2025; Kipf and Welling, 2017; Zeng et al., 2023) explore graph-based control without adaptive heuristic-learned velocity fusion.

Our work isolates spatial reasoning granularity and schedule smoothness as independent design dimensions within the hybrid recovery paradigm. Whereas prior studies evaluate hybrid modules as monolithic packages, we decompose the SCC design space along decision paradigm, spatial granularity, and trade-off scheduling. By holding the training objective and graph backbone fixed while systematically varying spatial granularity (node-level global vs. subnetwork-level rigid translation) and decoupling training-time versus inference-time fusion schedules, we identify the minimal architectural and dynamical conditions required for resilient recovery.

3 Preliminaries and Problem Formulation

3.1 UAV Swarm Network Model

The surviving UAV swarm is modeled as a time-varying proximity graph over a bounded planar mission area. Following Peng et al. (2026), at discrete time step t, the swarm forms a temporal graph $G_t = (V_t, E_t, P_t)$, where $V_t = \{v_{1,t}, \ldots, v_{|V_t|,t}\}$ denotes the surviving UAVs, E_t denotes the active communication links, and $P_t = \{\mathbf{p}_{1,t}, \ldots, \mathbf{p}_{|V_t|,t}\}$ with $\mathbf{p}_{i,t} \in \mathcal{A} \subset \mathbb{R}^2$ represents their planar coordinates. Let $\mathbf{A}_t$ be the binary adjacency matrix and $\mathbf{L}_t = \mathbf{D}_t - \mathbf{A}_t$ be the graph Laplacian. The number of connected components C_t equals the algebraic multiplicity of the zero eigenvalue of $\mathbf{L}_t$, and global connectivity holds if and only if $C_t = 1$.

Communication links and ground coverage follow Euclidean disk models governed by air-to-air path loss. An undirected link $(v_i, v_j) \in E_t$ exists if and only if the inter-UAV distance $d_{ij,t} = \|\mathbf{p}_{i,t} - \mathbf{p}_{j,t}\|_2$ satisfies $d_{ij,t} \leq R_c$, where R_c is the communication range derived from free-space air-to-air channel models (Ma et al., 2019; Ni et al., 2024; Wang et al., 2025a). Each UAV covers a circular disk $\mathcal{C}_i(t)$ of radius R_{cov}, yielding a collective swarm coverage ratio $C_{\text{cov}}(t) = \left|\bigcup_{v_i \in V_t} \mathcal{C}_i(t)\right| / |\mathcal{A}|$.

3.2 Optimization Objective and Kinematic Constraints

The self-healing connectivity recovery objective jointly penalizes reconnection delay and post-recovery coverage loss. Let t_1 denote the failure onset step and $t_r = \min\{t \geq t_1 : C_t = 1\}$ denote the earliest step at which full connectivity is restored within horizon T, giving a normalized recovery time ratio $T_{\text{rec}} = (t_r - t_1)/T$. We seek a velocity control policy $\hat{V}_t = \{\hat{\mathbf{v}}_{1,t}, \ldots, \hat{\mathbf{v}}_{|V_t|,t}\}$ that minimizes the composite cost

$$J_c = T_{\text{rec}} + \big(1 - C_{\text{cov}}(t_r)\big), \tag{1}$$

subject to four operational constraints: (C1) terminal connectivity $C_{t_r} = 1$; (C2) speed and arena bounds $\|\hat{\mathbf{v}}_{i,t}\|_2 \leq \hat{v}_{\max}$ and $\mathbf{p}_{i,t} \in \mathcal{A}$; (C3) discrete-time Euler kinematics $\mathbf{p}_{i,t} = \mathbf{p}_{i,t_1} + \sum_{t_1 \leq k < t} \Delta t \cdot \hat{\mathbf{v}}_{i,k}$; and (C4) link feasibility $d_{ij,t} \leq R_c$ for all active intra-swarm transmissions. Minimizing T_{rec} requires inward convergence, whereas minimizing $1 - C_{\text{cov}}(t_r)$ requires spatial dispersion.

3.3 A Three-Axis Taxonomy of the Design Space

Every self-healing recovery method can be categorized along three orthogonal design axes: decision paradigm, spatial granularity, and trade-off strategy.

- **Axis 1 (Decision Paradigm):** Motion commands are generated by a *heuristic* rule with zero learnable parameters, a *learned* parametric network trained via data or differentiable simulation, or a *hybrid* controller that explicitly fuses heuristic and learned outputs.
- **Axis 2 (Spatial Reasoning Granularity):** Control reasoning operates at the *node-level* using local neighborhood states, the *subnetwork-level* where each connected component acts as a rigid or coordinated unit, or the *global-level* where all surviving agents exchange context across the entire arena.
- **Axis 3 (Connectivity–Coverage Trade-Off Strategy):** Competing objectives are balanced via a *fixed* constant weight, a *staged* sequential switch, or a continuous *adaptive* schedule parameterized by time or topological state.

Table 1 maps representative prior methods and our four experimental architectures (GAC, SAC, HAC, and HSC) into this $3 \times 3 \times 3$ combinatorial space.

Table 1: Positioning of prior SCC methods and the architectures evaluated in this work within the $3 \times 3 \times 3$ design taxonomy.

Paradigm	Granularity	Trade-off	Representative method(s)
Heuristic	Node-level	Fixed	Akkaya et al. (2009); Mi and Yang (2015); Wang et al. (2024c)
Heuristic	Subnetwork	Fixed	Chen et al. (2020)
Heuristic	Global	Fixed	Lin and Ding (2025)
Heuristic	Global	Staged	HERO (Mi et al., 2011)
Learned	Node-level	Fixed	Mou et al. (2022); **GAC** (Section 4)
Learned	Subnetwork	Fixed	Lin et al. (2024); **SAC** (Section 4)
Learned	Global	Fixed	Guo et al. (2024)
Hybrid	Node-level	Fixed	Li et al. (2025)
Hybrid	Node+Subnet	Adaptive	R2C (ISE+IST+AWF) (Peng et al., 2026)
Hybrid	**Global**	**Fixed / Adaptive**	**HAC (this work, Section 5)**
Hybrid	**Subnetwork**	**Fixed / Adaptive**	**HSC (this work, Section 9)**

4 Motivating Experiments: Failure Modes of Single-Branch Designs

To motivate our hybrid architecture, we first isolate two purely learned baselines that share identical loss functions and network backbones but operate at opposite spatial granularities: node-level global convolution (GAC) and subnetwork-level rigid translation (SAC).

4.1 Global-Level Learned Policy (GAC)

A purely learned global graph policy (GAC) achieves moderate reconnection capability but suffers from severe spatial collapse. GAC constructs a k-nearest-neighbor (k-NN) graph over the Euclidean coordinates of all surviving UAVs regardless of communication range R_c, allowing message passing across disconnected components. A 4-layer GCN with hidden dimension 128 and a two-layer MLP head directly outputs per-node 2D velocities scaled via tanh by $\hat{v}_{\max}$ (without the unit-norm velocity projection used in hybrid fusion), trained end-to-end with equal weights on connectivity and coverage surrogate losses. As shown in Table 2, GAC's recovery success rate drops from 49.0 ± 0.6 out of 50 trials at 20% failure to 19.8 ± 5.5 at 80% failure, accompanied by an elevated average node degree (AND $= 13.68 \pm 0.28$ at 20% failure) that reflects severe spatial clustering around high-degree nodes.

4.2 Subnetwork-Level Rigid Translation (SAC)

Restricting the learned policy to subnetwork-level rigid translation (SAC) prevents spatial clustering but causes deterministic recovery collapse under moderate-to-high failure rates. SAC partitions surviving UAVs into connected components under R_c, pools each component into its centroid position and relative size, and applies the same 4-layer GCN over a k-NN graph of component centroids to predict a single rigid 2D translation vector shared by all members of each subnetwork. Table 2 demonstrates that across five random seeds, SAC maintains a low, uniform node degree (AND $= 6.71 \pm 0.09$ at 20% failure and 5.32 ± 0.18 at 40% failure), eliminating intra-cluster collapse. However, its recovery success rate drops to 29.4 ± 3.4 at 20% failure, 8.8 ± 1.9 at 40% failure, and collapses deterministically to 0.0 ± 0.0 out of 50 at both 60% and 80% failure

Table 2: Motivating baseline comparison under random deployment (mean ± std over 5 seeds, 50 trials per seed). GAC suffers from spatial clustering (high AND), whereas SAC avoids clustering but collapses deterministically at 60%–80% failure.

Failure	GAC (global node-level)		SAC (subnetwork rigid)	
Rate	AND	Success/50	AND	Success/50
20%	13.68 ± 0.28	49.0 ± 0.6	6.71 ± 0.09	29.4 ± 3.4
40%	12.48 ± 0.61	47.2 ± 1.7	5.32 ± 0.18	8.8 ± 1.9
60%	10.55 ± 0.78	42.8 ± 3.7	n/a	0.0 ± 0.0
80%	6.00 ± 0.26	19.8 ± 5.5	n/a	0.0 ± 0.0

across all five seeds. Rigid component translation lacks the internal degrees of freedom needed to simultaneously bridge multiple scattered singletons without tearing existing links.

4.3 Complementary Failure Modes Motivate Node-Level Hybridization

The opposite failure modes of GAC and SAC demonstrate that resilient recovery requires both node-level movement flexibility and an explicit local spacing prior. GAC retains full $2|V_t|$ spatial degrees of freedom to bridge fragmented clusters but lacks a short-range repulsive mechanism to prevent over-concentration. Conversely, SAC preserves internal spacing by freezing relative intra-component positions but loses the ability to deform components toward multiple targets. This complementarity directly motivates fusing a parameter-free local virtual-force field with a node-level global GCN policy.

5 Method: Hybrid Adaptive Convolution (HAC)

5.1 Architecture Overview

Hybrid Adaptive Convolution (HAC) fuses a parameter-free buffered virtual-force heuristic with a single global graph convolutional network at the individual node velocity level. Unlike multipartite subnetwork architectures that require Quickhull boundary extraction and cross-subgraph virtual edge construction, HAC operates directly on the homogeneous set of surviving UAVs. This streamlined design isolates the core benefit of heuristic–learned velocity coupling from complex geometric preprocessing.

5.2 Heuristic Branch: Buffered Three-Zone Virtual-Force Field

The heuristic branch regulates local inter-UAV spacing and arena containment through a parameter-free three-zone potential field. Adopting the buffered virtual-spring formulation (Rychlewski, 1984; Peng et al., 2026), we partition the communication disk $(0, R_c]$ around an equilibrium buffer zone $[d_l, d_u] = [0.8R_c, 0.9R_c]$. The pairwise virtual force exerted on UAV v_i by a communicating neighbor $v_j \in \mathcal{N}_i = \{v_j \in V_t : 0 < d_{ij,t} \leq R_c\}$ is defined as

$$\mathbf{F}'_{ij,t} = \begin{cases} -\exp\left(-\alpha \cdot \dfrac{d_{ij,t}}{d_l}\right) \mathbf{u}_{ij,t}, & 0 \leq d_{ij,t} < d_l \\ \mathbf{0}, & d_l \leq d_{ij,t} \leq d_u \\ \exp\left(-\beta \cdot \dfrac{R_c - d_{ij,t}}{R_c - d_u}\right) \mathbf{u}_{ij,t}, & d_u < d_{ij,t} \leq R_c, \end{cases} \tag{2}$$

where $\mathbf{u}_{ij,t} = (\mathbf{p}_{j,t} - \mathbf{p}_{i,t})/d_{ij,t}$ is the unit direction vector from v_i to v_j, and $\alpha = \beta = 0.01$ govern the exponential decay rates in the repulsive zone $[0, d_l)$ and attractive buffer zone $(d_u, R_c]$. To prevent agents from exiting the mission area $\mathcal{A}$, any UAV within distance $d_{i,k} < d_e = 0.15R_c$ of boundary edge $k \in \mathcal{B} = \{\text{left, right, bottom, top}\}$ experiences an inward normal repulsion $\mathbf{F}''_{i,t} = \sum_{k\in\mathcal{B}} \exp(-\delta \cdot d_{i,k})\, \mathbf{n}_k$ with $\delta = 0.01$. Summing $\mathbf{F}_{i,t} = \sum_{j\in\mathcal{N}_i} \mathbf{F}'_{ij,t} + \mathbf{F}''_{i,t}$ and normalizing yields the heuristic velocity command $\mathbf{v}'_{i,t} = \hat{v}_{\max} \cdot \mathbf{F}_{i,t}/(\|\mathbf{F}_{i,t}\|_2 + \epsilon)$. Because Eq. (2) contains no trainable weights, $\mathbf{v}'_{i,t}$ provides an invariant physical anchor throughout both training and inference.

5.3 Learned Branch and Velocity-Level Adaptive Fusion

The learned branch propagates topological context across disconnected components via a global k-NN graph convolutional network. At each step t, we construct a symmetric k-NN graph ($k = 8$) over the Euclidean coordinates of all surviving UAVs V_t. Each node v_i receives a 3-dimensional input feature vector comprising its normalized planar coordinates and its normalized k-NN degree. This feature representation passes through an $L = 4$ layer GCN with hidden dimension 128 and ReLU activations, followed by a two-layer MLP head with tanh activation scaled by $\hat{v}_{\max}$ to produce the learned velocity vector $\mathbf{v}''_{i,t}$.

Velocity-level fusion combines heuristic and learned commands through a tunable sigmoid schedule followed by unit-speed renormalization. To govern the temporal handover from heuristic expansion to learned global translation, we define the time-dependent gating weights

$$w_T(t) = \frac{1}{1 + \exp\big(-k \cdot (t/T - s_0)\big)}, \qquad w_E(t) = 1 - w_T(t), \tag{3}$$

where $k > 0$ controls transition steepness and $s_0 \in (0, 1)$ specifies the crossover midpoint relative to horizon T. The executed velocity command $\hat{\mathbf{v}}_{i,t}$ is obtained by renormalizing the weighted sum:

$$\hat{\mathbf{v}}_{i,t} = \hat{v}_{\max} \cdot \frac{w_E(t)\, \mathbf{v}'_{i,t} + w_T(t)\, \mathbf{v}''_{i,t}}{\left\| w_E(t)\, \mathbf{v}'_{i,t} + w_T(t)\, \mathbf{v}''_{i,t} \right\|_2 + \epsilon}. \tag{4}$$

We evaluate three primary configurations of Eq. (4): **HAC-Fixed**, which sets constant equal weights $w_E(t) = w_T(t) = 0.5$; **HAC-Adaptive**, which varies (k, s_0) in Eq. (3); and the **Learned-only** ablation, which sets $w_T(t) = 1$ and $w_E(t) = 0$ inside Eq. (4) so that $\mathbf{v}''_{i,t}$ undergoes exact unit-norm projection $\hat{v}_{\max}\mathbf{v}''_{i,t}/(\|\mathbf{v}''_{i,t}\|_2 + \epsilon)$ (distinguishing it from GAC, which uses unprojected tanh scaling).

5.4 Cardinality-Invariant Training Objective and Algorithms

A cardinality-invariant soft-minimum connectivity surrogate eliminates component-size bias during differentiable multi-step rollout training. Because the discrete component count C_t is non-differentiable, we supervise inter-component bridging using a smooth soft-minimum distance surrogate over every pair of currently disconnected components S_i, S_j:

$$\mathcal{L}^{(i,j)}_{\text{conn}} = -\tau \left(\log \sum_{\substack{u\in S_i \\ v\in S_j}} \exp\left(-\frac{d_{uv}}{\tau}\right) - \log(n_i n_j) \right), \tag{5}$$

averaged over all disconnected component pairs, where $n_i = |S_i|$, $n_j = |S_j|$, and $\tau > 0$ is a temperature parameter. Standard log-sum-exp soft-minimum estimators omit the $-\log(n_i n_j)$

Algorithm 1 HAC Differentiable Rollout Training (One Epoch)

Require: Learned policy π_θ, training rollout length T_{train}, training schedule $w_T^{\text{train}}(\cdot)$

Sample failure ratio $r \sim \{0.2, 0.4, 0.6, 0.8\}$, initialize positions $\mathbf{P}_0$, and remove failed nodes
$\mathcal{L}_{\text{total}} \leftarrow 0$
for $t = 0, \ldots, T_{\text{train}} - 1$ **do**
    Identify connected components $\{S_m\}$ of $\mathbf{P}_t$ under communication range R_c
    Compute heuristic velocities $\mathbf{v}'$ via Eq. (2) ▷ detached, zero trainable parameters
    Construct global k-NN graph over $\mathbf{P}_t$ and evaluate $\mathbf{v}'' \leftarrow \pi_\theta(\mathbf{P}_t)$ ▷ learned branch
    Evaluate gating weights $w_T \leftarrow w_T^{\text{train}}(t)$ and $w_E \leftarrow 1 - w_T$
    Fuse velocities $\hat{\mathbf{v}} \leftarrow \text{Fuse}(\mathbf{v}', \mathbf{v}'', w_E, w_T)$ via Eq. (4)
    Update positions $\mathbf{P}_{t+1} \leftarrow \text{clip}(\mathbf{P}_t + \Delta t \cdot \hat{\mathbf{v}},\ 0,\ L)$
    $\mathcal{L}_{\text{total}} \leftarrow \mathcal{L}_{\text{total}} + 0.5\,\mathcal{L}_{\text{conn}}(\mathbf{P}_{t+1}) + 0.5\,\mathcal{L}_{\text{cov}}(\mathbf{P}_{t+1}) + \lambda\,\mathcal{L}_{\text{move}}$
end for
Backpropagate $\mathcal{L}_{\text{total}}/T_{\text{train}}$ through π_θ and update θ via Adam with gradient clipping

Algorithm 2 HAC Inference Deployment Rollout

Require: Trained policy π_θ, evaluation horizon T, evaluation schedule $w_T^{\text{eval}}(\cdot)$

for $t = 0, \ldots, T - 1$ **do**
    if $C_t = 1$ (swarm is fully connected) **then return** success, recovery step t
    end if
    Compute heuristic velocities $\mathbf{v}'$ via Eq. (2)
    Evaluate learned velocities $\mathbf{v}'' \leftarrow \pi_\theta(\mathbf{P}_t)$ ▷ no gradient computation
    Fuse velocities $\hat{\mathbf{v}} \leftarrow \text{Fuse}(\mathbf{v}', \mathbf{v}'', 1 - w_T^{\text{eval}}(t), w_T^{\text{eval}}(t))$ via Eq. (4)
    Update positions $\mathbf{P}_{t+1} \leftarrow \text{clip}(\mathbf{P}_t + \Delta t \cdot \hat{\mathbf{v}},\ 0,\ L)$
end for
return failure (horizon T exhausted)

correction, introducing an additive distortion of $-\tau \log(n_i n_j)$ that systematically under-penalizes large components and yields spuriously negative loss values (verified numerically in Appendix A). Including $-\log(n_i n_j)$ ensures exact cardinality invariance so that isolated singletons receive gradient magnitudes consistent with large subnetworks.

Spatial coverage and movement efficiency are supervised jointly through differentiable grid-coverage and displacement regularizers. We approximate area coverage over a uniform grid $G \subset \mathcal{A}$ via $\mathcal{L}_{\text{cov}} = \frac{1}{|G|} \sum_{\mathbf{g} \in G} \big(1 - \sigma\big((R_{\text{cov}} - \min_i \|\mathbf{g} - \mathbf{p}_i\|_2)/\tau'\big)\big)$ and penalize excessive control effort via $\mathcal{L}_{\text{move}} = \frac{1}{|V_t|} \sum_i \|\mathbf{p}_{i,t+1} - \mathbf{p}_{i,t}\|_2$. Throughout training, connectivity and coverage losses receive equal static weights of 0.5, ensuring that all temporal trade-offs occur strictly at the velocity fusion layer in Eq. (4). Algorithms 1 and 2 summarize end-to-end differentiable rollout training and closed-loop deployment.

6 Theoretical Analysis

Section 9 will show empirically that applying a steep adaptive schedule ($k = 30, s_0 = 0.125$) to HAC degrades recovery success under 80% failure, whereas reducing the transition slope k restores robustness. We analyze two candidate mechanisms for this phenomenon: an inference-time link-retention barrier in the heuristic branch (Section 6.1) and a training-time implicit curriculum

governed by autonomy exposure (Section 6.2), which we subsequently adjudicate via decoupled cross-evaluation in Section 9.3.

6.1 Link-Retention Barrier of the Heuristic Branch

Proposition 1 (Link-retention barrier in the attractive buffer zone)**.** *Let v_i, v_j be two connected UAVs occupying the attractive buffer zone $d_u < d_{ij,t} \le R_c$ of Eq. (2), and suppose no external repulsive or boundary force acts on v_i or v_j. Then, for any step size $0 < \Delta t < d_{ij,t}/(2\hat{v}_{\max})$, executing the heuristic branch alone ($w_E = 1, w_T = 0$ in Eq. (4)) strictly decreases their inter-UAV distance according to $d_{ij,t+\Delta t} = d_{ij,t} - 2\Delta t\,\hat{v}_{\max} < R_c$, thereby guaranteeing that the existing communication link $(v_i, v_j) \in E_t$ cannot break at step $t + \Delta t$.*

Proof. By Eq. (2), the pairwise force on v_i from v_j within $d_u < d_{ij,t} \le R_c$ is $\mathbf{F}'_{ij,t} = \exp\big(-\beta(R_c - d_{ij,t})/(R_c - d_u)\big)\mathbf{u}_{ij,t}$ pointing from v_i toward v_j. By symmetry, $\mathbf{F}'_{ji,t} = -\mathbf{F}'_{ij,t}$ points from v_j toward v_i with equal magnitude. Under the isolation assumption, $\mathbf{F}_{i,t} = \mathbf{F}'_{ij,t}$ and $\mathbf{F}_{j,t} = -\mathbf{F}'_{ij,t}$, which after unit-speed normalization yields $\mathbf{v}'_{i,t} = \hat{v}_{\max}\,\mathbf{u}_{ij,t}$ and $\mathbf{v}'_{j,t} = -\hat{v}_{\max}\,\mathbf{u}_{ij,t}$. Applying the one-step Euler update gives $d_{ij,t+\Delta t} = \|\mathbf{p}_{i,t} + \Delta t\,\mathbf{v}'_{i,t} - \mathbf{p}_{j,t} - \Delta t\,\mathbf{v}'_{j,t}\|_2 = \|(\mathbf{p}_{i,t} - \mathbf{p}_{j,t}) + 2\Delta t\,\hat{v}_{\max}\,\mathbf{u}_{ij,t}\|_2$. Substituting $\mathbf{p}_{i,t} - \mathbf{p}_{j,t} = -d_{ij,t}\,\mathbf{u}_{ij,t}$ yields $d_{ij,t+\Delta t} = |d_{ij,t} - 2\Delta t\,\hat{v}_{\max}| = d_{ij,t} - 2\Delta t\,\hat{v}_{\max} < d_{ij,t} \le R_c$ for all $0 < \Delta t < d_{ij,t}/(2\hat{v}_{\max})$. □

Remark 1 (Inference-time handover risk under long horizons). Proposition 1 establishes that the attractive buffer zone $(d_u, R_c]$ acts as a local restorative barrier that prevents existing communication links from fracturing, while the repulsive zone $[0, d_l)$ drives intra-subnetwork expansion to prevent over-clustering. Because the learned branch $\mathbf{v}''_{i,t}$ carries no structural barrier guarantee and exhibits inward aggregation bias, driving $w_E(t) \to 0$ too early during a 400-step inference rollout removes the restorative force buffer, allowing imperfect learned velocities to break newly formed bridges and over-cluster surviving nodes.

6.2 Autonomy Exposure and Decoupled Schedule Testability

A competing training-time hypothesis posits that steep schedules degrade policy learning by inflating autonomy exposure during differentiable rollouts, a claim that can be directly falsified by decoupling training and evaluation schedules. Because π_θ is trained end-to-end through Eq. (4) in Algorithm 1, the schedule profile $w_T(t)$ determines the fraction of rollout steps in which the learned branch operates without heuristic assistance.

Definition 1 (Autonomy exposure)**.** For a rollout of length H and dominance threshold $\tau \in (0, 1)$, the *autonomy exposure* of schedule $w_T(\cdot; k, s_0)$ is the fraction of steps during which the learned branch holds majority control:

$$\mathrm{AE}_H(k, s_0; \tau) = \frac{1}{H}\Big|\big\{\, t \in \{0, \ldots, H-1\} : w_T(t; k, s_0) > \tau \,\big\}\Big|. \tag{6}$$

For $\tau = 0.5$, inverting Eq. (3) between $w_T = 0.1$ and $w_T = 0.9$ yields a transition window of width $\Delta t \approx \frac{\ln(81)}{k} H \approx \frac{4.394}{k} H$. Under a steep slope ($k = 30, s_0 = 0.125$), $w_T(t)$ surges past 0.9 within the first 20% of the horizon, whereas a gentle slope ($k = 5, s_0 = 0.7$) maintains $w_E(t) > 0.3$ across the vast majority of the rollout.

Remark 2 (Adjudicating training-time vs. inference-time mechanisms). If steep schedules fail primarily because high training-time autonomy exposure $\mathrm{AE}_{T_{\text{train}}}$ corrupts gradient supervision (the *implicit training curriculum hypothesis*), then a policy trained under a steep schedule ($w_T^{\text{train}} = \text{Steep}$) should

remain defective even when evaluated under a fixed or smooth schedule ($w_T^{\text{eval}} \in \{\text{Fixed}, \text{Smooth}\}$). Conversely, if the failure is governed by inference-time loss of the heuristic link-retention barrier over the 400-step deployment horizon (the *inference-time barrier hypothesis* of Proposition 1), then performance will depend strictly on w_T^{eval} regardless of w_T^{train}. Section 9.3 executes this exact 3×3 decoupled experiment across five random seeds, falsifying the training-corruption account and confirming the inference-time barrier mechanism.

7 Experimental Setup

7.1 Simulation Environment and Multi-Seed Protocol

All architectures and ablations are evaluated across five independent random seeds under random and uniform swarm deployments up to 80% node failure. Following the benchmark protocol of Peng et al. (2026), we simulate a $1000 \times 1000\,\text{m}^2$ mission area under two initial layouts: **random deployment** with 200 UAVs sampled uniformly at random, and **uniform deployment** with 196 UAVs initialized on a regular 14×14 grid. Each UAV operates with communication and coverage radii $R_c = R_{\text{cov}} = 120\,\text{m}$, maximum speed $\hat{v}_{\max} = 1\,\text{m/s}$, time step $\Delta t = 1\,\text{s}$, and an evaluation horizon of $T = 400$ steps. We test four failure ratios $r \in \{20\%, 40\%, 60\%, 80\%\}$ via uniform random node removal. Every configuration in every table of this paper is evaluated over 5 independent training seeds $\times$ 50 independent evaluation trials per seed (250 trials per cell), reporting the cross-seed mean and standard deviation.

7.2 Baselines and Evaluation Metrics

We benchmark HAC against purely heuristic, purely learned, subnetwork-level rigid, and published multipartite baselines across five performance and efficiency metrics. Our controlled baselines comprise: (i) **Heuristic-only**, executing Eq. (2) in isolation ($w_E = 1, w_T = 0$); (ii) **GAC-Fixed** and **SAC-Fixed** from Section 4; (iii) **Learned-only**, setting $w_T = 1, w_E = 0$ in Eq. (4); and (iv) **HSC**, substituting SAC's subnetwork-level rigid-translation GCN into the hybrid fusion loop. We also compare against the published figures of **R2C** (Peng et al., 2026). Performance is quantified via five metrics:

- **Success/50 (↑):** Number of evaluation trials out of 50 that restore full connectivity ($C_t = 1$) within $T = 400$ steps. Non-recovering trials are excluded from conditional post-recovery averages.
- **REC (↓):** Normalized recovery time ratio $T_{\text{rec}} = (t_r - t_1)/T$ averaged over successful trials.
- **COV (↑):** Swarm coverage ratio $C_{\text{cov}}(t_r)$ at the instant of reconnection, evaluated via 5 m grid sampling.
- **AND:** Average node degree at t_r, serving as an inverse proxy for spatial dispersion (elevated AND indicates spatial clustering).
- **Runtime (↓):** Mean wall-clock execution time in seconds per evaluation trial.

7.3 Implementation Details

All learned policies use an identical 4-layer GCN backbone with full-pipeline deterministic seeding. Every graph convolutional model uses $L = 4$ layers, hidden dimension 128, $k = 8$ nearest neighbors, movement cost weight $\lambda = 0.01$, and Adam optimization with learning rate 10^{-3} and gradient norm clipping at 1.0 over 300 epochs of $T_{\text{train}} = 40$ differentiable rollout steps. Python,

Table 3: Main benchmark results under random deployment (mean ± std over 5 seeds, 50 trials per seed). Here n/a denotes zero successful trials.

Architecture	Failure	Success/50 ↑	REC ↓	COV ↑	Runtime (s) ↓
HAC-Fixed	20%	50.0 ± 0.0	0.051 ± 0.011	0.997 ± 0.000	0.384 ± 0.041
	40%	50.0 ± 0.0	0.134 ± 0.005	0.988 ± 0.001	0.511 ± 0.027
	60%	50.0 ± 0.0	0.331 ± 0.009	0.928 ± 0.008	0.799 ± 0.040
	80%	$\mathbf{46.6 \pm 3.9}$	0.696 ± 0.048	0.602 ± 0.009	$\mathbf{1.282 \pm 0.108}$
HAC-Smooth $(5, 0.7)$	20%	49.8 ± 0.4	0.034 ± 0.008	0.997 ± 0.000	0.336 ± 0.022
	40%	50.0 ± 0.0	0.110 ± 0.007	0.993 ± 0.000	0.437 ± 0.020
	60%	49.6 ± 0.5	0.325 ± 0.020	0.949 ± 0.004	0.779 ± 0.034
	80%	43.6 ± 2.8	$\mathbf{0.684 \pm 0.020}$	$\mathbf{0.616 \pm 0.006}$	1.291 ± 0.043
GAC-Fixed	20%	49.0 ± 0.6	0.324 ± 0.007	0.747 ± 0.009	1.520 ± 0.110
	40%	47.2 ± 1.7	0.467 ± 0.020	0.614 ± 0.021	1.890 ± 0.130
	60%	42.8 ± 3.7	0.612 ± 0.027	0.452 ± 0.022	2.140 ± 0.120
	80%	19.8 ± 5.5	0.775 ± 0.017	0.310 ± 0.009	1.910 ± 0.080
SAC-Fixed	20%	29.4 ± 3.4	0.232 ± 0.047	0.942 ± 0.014	2.240 ± 0.260
	40%	8.8 ± 1.9	0.577 ± 0.068	0.842 ± 0.037	2.722 ± 0.205
	60%	0.0 ± 0.0	n/a	n/a	2.211 ± 0.163
	80%	0.0 ± 0.0	n/a	n/a	1.778 ± 0.125
Learned-only	20%	45.6 ± 2.9	0.212 ± 0.025	0.901 ± 0.014	1.712 ± 0.322
	40%	37.8 ± 3.5	0.468 ± 0.030	0.719 ± 0.025	2.585 ± 0.184
	60%	26.4 ± 2.8	0.702 ± 0.020	0.503 ± 0.024	2.467 ± 0.082
	80%	8.6 ± 3.3	0.831 ± 0.023	0.347 ± 0.023	1.894 ± 0.057
Heuristic-only	20%	47.2 ± 1.2	0.045 ± 0.015	0.997 ± 0.000	0.363 ± 0.040
	40%	30.8 ± 1.3	0.125 ± 0.021	0.993 ± 0.001	0.625 ± 0.020
	60%	1.4 ± 1.4	0.227 ± 0.092	0.964 ± 0.007	0.784 ± 0.028
	80%	0.0 ± 0.0	n/a	n/a	0.551 ± 0.021

NumPy, CPU PyTorch (`torch.manual_seed`), and CUDA PyTorch (`torch.cuda.manual_seed_all`) generators are strictly seeded prior to both model construction and environment reset.

8 Results

8.1 Main Results: HAC vs. Single-Branch Baselines

HAC-Fixed and HAC-Smooth achieve near-perfect connectivity recovery, superior coverage retention, and 1.5× to 2.2× faster execution across both deployments, outperforming all single-branch baselines by wide margins under 80% node failure. Tables 3 and 4 report the five-seed benchmark results for random and uniform deployments, and Figure 1 visualizes the random-deployment recovery trajectories. Under 20% to 60% node loss, HAC-Fixed achieves a 100% recovery rate (50.0 ± 0.0 out of 50) in both scenarios. Under extreme 80% failure, HAC-Fixed succeeds in 46.6 ± 3.9 trials (random) and 49.4 ± 0.8 trials (uniform), while HAC-Smooth ($k = 5, s_0 = 0.7$) succeeds in 43.6 ± 2.8 (random) and 47.8 ± 1.3 (uniform) trials. Both hybrid variants outperform the strongest single-branch baseline (GAC-Fixed at 19.8 ± 5.5 and 25.8 ± 6.1) by 23.8 to 26.8 successful trials while nearly doubling spatial coverage (COV of 0.602–0.616 vs. 0.310 in random deployment, an improvement of 29.2 to 30.6 pp). Furthermore, HAC-Fixed completes 80%-failure episodes in 1.21–1.28 s (roughly 3 ms per step), running 1.5× faster than Learned-only (1.89–1.94 s) and SAC-Fixed (1.78–1.85 s).

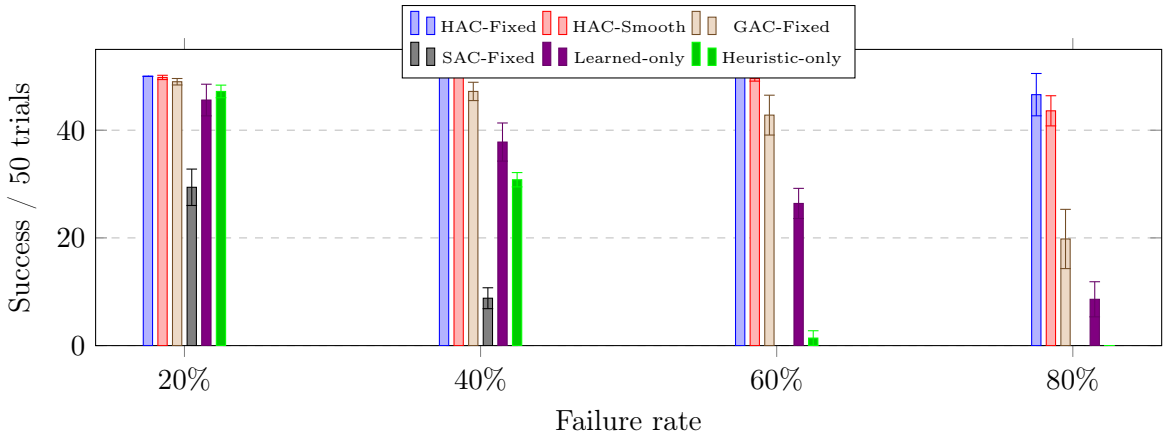


Figure 1: Connectivity recovery success count (out of 50 trials) across node failure rates under random deployment (mean ± std over 5 seeds). Both HAC-Fixed and HAC-Smooth sustain high recovery rates across all failure regimes.

8.2 Comparison with Multipartite Subnetwork Architectures

HAC-Fixed and HAC-Smooth retain 88.4% to 91.2% of the coverage reported by R2C under 80% node failure despite omitting convex-hull boundary extraction, virtual multipartite edges, and spectral graph filtering. Table 5 compares HAC-Fixed against the published figures of R2C (Peng et al., 2026). At 20% and 40% failure rates, both methods achieve virtually identical coverage (≥ 0.988), with HAC-Fixed achieving faster recovery under uniform deployment (REC of 0.005 vs. 0.037 at 20% failure and 0.040 vs. 0.051 at 40% failure). Under the most severe 80% failure regime, R2C's reported coverage exceeds that of HAC-Fixed by an absolute margin of 7.9 pp in random deployment (0.681 vs. 0.602, or 6.5 pp vs. HAC-Smooth's 0.616) and 5.9 pp in uniform deployment (0.668 vs. 0.609), meaning that HAC-Fixed retains 88.4% (random, 90.5% for HAC-Smooth) and 91.2% (uniform) of R2C's coverage. Considering that R2C relies on Quickhull boundary extraction, multipartite cross-subnetwork edge construction, and an 8-layer, 512-dimensional spectral GCN, whereas HAC uses a single global 4-layer, 128-dimensional GCN, these results confirm that the primary resilience gain stems from coupling a local virtual-force prior with a global node-level learned policy rather than complex subnetwork partitioning.

8.3 Topology Quality and Anti-Clustering Analysis

HAC-Fixed and HAC-Smooth completely eliminate the GNN spatial-collapse failure mode while sustaining high topological reconnection. Figure 2 plots the five-seed post-recovery average node degree (AND) across failure rates under random deployment. Whereas GAC-Fixed and Learned-only exhibit inflated node degrees (AND up to 13.68 and 10.12) caused by inward GNN aggregation bias, HAC-Fixed and HAC-Smooth track the low, smoothly decreasing degree profile of Heuristic-only and SAC-Fixed (AND declining from 6.73 and 6.63 at 20% failure to 2.58 and 2.48 at 80% failure). Velocity-level fusion with the buffered virtual-force field preserves uniform spatial dispersion without sacrificing global reconnection reach.

Table 4: Main benchmark results under uniform deployment (mean ± std over 5 seeds, 50 trials per seed).

Architecture	Failure	Success/50 ↑	REC ↓	COV ↑	Runtime (s) ↓
HAC-Fixed	20%	50.0 ± 0.0	0.005 ± 0.001	1.000 ± 0.000	0.238 ± 0.015
	40%	50.0 ± 0.0	0.040 ± 0.004	0.998 ± 0.000	0.269 ± 0.027
	60%	50.0 ± 0.0	0.249 ± 0.023	0.960 ± 0.004	0.649 ± 0.038
	80%	$\mathbf{49.4 \pm 0.8}$	$\mathbf{0.643 \pm 0.026}$	$\mathbf{0.609 \pm 0.004}$	$\mathbf{1.208 \pm 0.059}$
HAC-Smooth $(5, 0.7)$	20%	50.0 ± 0.0	0.005 ± 0.001	1.000 ± 0.000	0.233 ± 0.012
	40%	50.0 ± 0.0	0.039 ± 0.004	0.998 ± 0.000	0.255 ± 0.017
	60%	50.0 ± 0.0	0.252 ± 0.019	0.967 ± 0.001	0.633 ± 0.040
	80%	47.8 ± 1.3	0.665 ± 0.040	0.604 ± 0.007	1.262 ± 0.098
GAC-Fixed	20%	50.0 ± 0.0	0.011 ± 0.003	0.992 ± 0.003	0.350 ± 0.040
	40%	50.0 ± 0.0	0.200 ± 0.014	0.843 ± 0.019	0.980 ± 0.080
	60%	47.0 ± 2.8	0.574 ± 0.013	0.478 ± 0.023	1.850 ± 0.110
	80%	25.8 ± 6.1	0.772 ± 0.057	0.300 ± 0.012	1.920 ± 0.090
SAC-Fixed	20%	48.8 ± 1.2	0.007 ± 0.007	0.999 ± 0.001	0.341 ± 0.111
	40%	31.4 ± 3.3	0.161 ± 0.013	0.959 ± 0.007	1.492 ± 0.266
	60%	0.2 ± 0.4	0.003 ± 0.000	0.964 ± 0.000	2.212 ± 0.145
	80%	0.0 ± 0.0	n/a	n/a	1.851 ± 0.155
Learned-only	20%	49.6 ± 0.5	0.023 ± 0.015	0.988 ± 0.009	0.388 ± 0.115
	40%	48.4 ± 1.0	0.211 ± 0.014	0.892 ± 0.012	1.016 ± 0.095
	60%	36.4 ± 3.3	0.620 ± 0.068	0.569 ± 0.048	2.065 ± 0.118
	80%	11.2 ± 3.7	0.839 ± 0.039	0.344 ± 0.019	1.937 ± 0.099
Heuristic-only	20%	49.8 ± 0.4	0.006 ± 0.002	1.000 ± 0.000	0.228 ± 0.022
	40%	44.0 ± 1.3	0.078 ± 0.017	0.998 ± 0.000	0.340 ± 0.032
	60%	0.8 ± 0.7	0.110 ± 0.081	0.977 ± 0.009	0.776 ± 0.057
	80%	0.0 ± 0.0	n/a	n/a	0.550 ± 0.027

9 Ablation Studies

9.1 Synergistic Branch Contribution

Velocity-level hybrid fusion exhibits strict super-additive synergy under severe fragmentation rather than merely selecting the better constituent branch. Comparing HAC-Fixed against its isolated branches in Tables 3 and 4 rules out simple branch dominance. Under random deployment at 80% failure, Heuristic-only succeeds in 0.0 ± 0.0 trials and Learned-only succeeds in 8.6 ± 3.3 trials, yet their 50/50 velocity fusion achieves 46.6 ± 3.9 successes (exceeding the sum of both constituent branches by 38.0 trials). Similarly, under uniform deployment at 80% failure, fusion achieves 49.4 ± 0.8 successes compared to 0.0 ± 0.0 (Heuristic-only) and 11.2 ± 3.7 (Learned-only). The heuristic branch prevents local link breakage and spatial collapse (Proposition 1), creating well-spaced local clusters that the global GCN branch steers into contact across long-range voids.

9.2 Five-Seed Adaptive Fusion Schedule Sweep

Across five random seeds, reducing the transition slope k monotonically improves 80%-failure recovery at every midpoint s_0, with the smooth schedule $(k = 5, s_0 = 0.7)$ restoring near-optimal robustness across both deployments. Table 6 reports the five-seed schedule sweep across slope $k \in \{5, 10, 30\}$ and midpoint $s_0 \in \{0.125, 0.5, 0.7\}$ under random deployment. At $s_0 = 0.125$, reducing slope from $k = 30$ to $k = 10$ increases 80%-failure recovery from 14.2 ± 4.3 to 18.4 ± 5.0 and raises coverage from 0.413 to 0.462. At $s_0 = 0.5$, reducing slope from $k = 30$ to $k = 10$ boosts 80%-failure recovery nearly sixfold from 5.2 ± 1.3 to 29.6 ± 4.5. Widening the transition

Table 5: Comparison between HAC-Fixed and published numbers of R2C (Peng et al., 2026). Lower REC ↓ and higher COV ↑ indicate superior recovery speed and spatial coverage.

	Random Deployment				Uniform Deployment			
Failure	R2C (Peng et al., 2026)		HAC-Fixed		R2C (Peng et al., 2026)		HAC-Fixed	
Rate	REC↓	COV↑	REC↓	COV↑	REC↓	COV↑	REC↓	COV↑
20%	0.023	0.996	0.051	0.997	0.037	1.000	0.005	1.000
40%	0.070	0.989	0.134	0.988	0.051	0.999	0.040	0.998
60%	0.204	0.951	0.331	0.928	0.181	0.978	0.249	0.960
80%	0.625	**0.681**	0.696	**0.602**	0.617	**0.668**	0.643	**0.609**

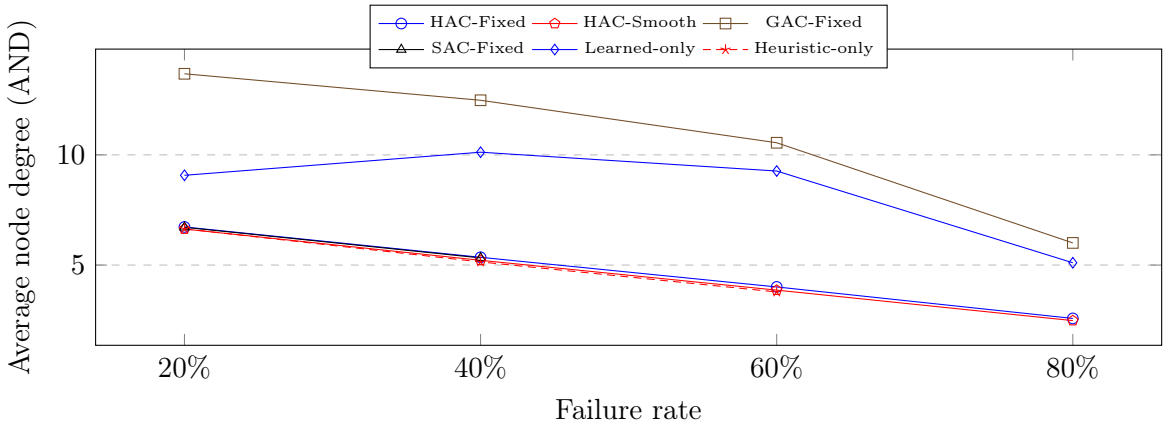


Figure 2: Average node degree (AND) across failure rates under random deployment (5-seed mean). HAC-Fixed and HAC-Smooth avoid the high-degree spatial collapse of GAC-Fixed and Learned-only while maintaining the highest recovery success rates.

window further to ($k = 5, s_0 = 0.7$) reaches 43.6 ± 2.8 successes with COV $= 0.616 \pm 0.006$ and AND $= 2.479 \pm 0.058$ in random deployment, and 47.8 ± 1.3 successes with COV $= 0.604 \pm 0.007$ in uniform deployment (Table 4). Notice that steep schedules ($k = 30$) also inflate post-recovery node degree (AND $= 4.114 \pm 0.296$ at $s_0 = 0.125$), confirming that premature suppression of the heuristic branch induces spatial clustering.

9.3 Decoupled Training vs. Inference Schedule Matrix

Cross-evaluating policies across all 3×3 pairings of training and evaluation schedules falsifies the training-corruption hypothesis and proves that steep-schedule collapse is strictly an inference-time handover phenomenon. Table 7 presents the five-seed 3×3 decoupled matrix crossing $w_T^{\text{train}} \in \{\text{Fixed}, \text{Smooth}\ (5, 0.7), \text{Steep}\ (30, 0.125)\}$ with $w_T^{\text{eval}} \in \{\text{Fixed}, \text{Smooth}\ (5, 0.7), \text{Steep}\ (30, 0.125)\}$ under random deployment at 60% and 80% failure. Two striking patterns emerge:

1. **Steep training does not impair the learned policy representation:** Models trained under the steep schedule ($w_T^{\text{train}} = \text{Steep}$) achieve 46.6 ± 0.5 successes when evaluated with Fixed weights and 46.4 ± 0.5 successes when evaluated with Smooth weights at 80% failure, accompanied by the highest coverage (0.622 ± 0.004 and 0.624 ± 0.003) and lowest cross-seed variance (± 0.5 trials) in the entire study. Because training rollouts span $T_{\text{train}} = 40$ steps, strong gradient supervision during early-to-mid rollout steps suffices to train an accurate global

Table 6: Five-seed adaptive fusion schedule sweep across slope k and midpoint s_0 under random deployment (mean ± std over 5 seeds, 50 trials per seed). At every midpoint s_0, gentler slopes (smaller k) strictly improve 80%-failure recovery and coverage.

	Success / 50 Trials ↑				Metrics at 80% Failure		
Schedule (k, s_0)	20%	40%	60%	80%	REC ↓	COV ↑	AND
Steep Early $(30, 0.125)$	49.4 ± 0.5	47.2 ± 1.6	41.0 ± 3.3	14.2 ± 4.3	0.796 ± 0.032	0.413 ± 0.032	4.114 ± 0.296
Moderate Early $(10, 0.125)$	49.8 ± 0.4	49.4 ± 0.8	46.0 ± 2.2	18.4 ± 5.0	0.746 ± 0.050	0.462 ± 0.036	3.426 ± 0.318
Steep Mid $(30, 0.5)$	49.8 ± 0.4	49.6 ± 0.5	42.8 ± 0.7	5.2 ± 1.3	0.855 ± 0.037	0.516 ± 0.020	2.604 ± 0.152
Moderate Mid $(10, 0.5)$	50.0 ± 0.0	50.0 ± 0.0	49.8 ± 0.4	29.6 ± 4.5	$\mathbf{0.673 \pm 0.016}$	0.571 ± 0.022	2.579 ± 0.075
Smooth Late $(5, 0.7)$	49.8 ± 0.4	50.0 ± 0.0	49.6 ± 0.5	43.6 ± 2.8	0.684 ± 0.020	$\mathbf{0.616 \pm 0.006}$	$\mathbf{2.479 \pm 0.058}$
HAC-Fixed $(50/50)$	$\mathbf{50.0 \pm 0.0}$	$\mathbf{50.0 \pm 0.0}$	$\mathbf{50.0 \pm 0.0}$	$\mathbf{46.6 \pm 3.9}$	0.696 ± 0.048	0.602 ± 0.009	2.578 ± 0.027

Table 7: Five-seed 3×3 decoupled training-versus-evaluation schedule matrix under random deployment (mean ± std over 5 seeds, 50 trials per seed). Collapse occurs exclusively when the *evaluation* schedule is Steep ($w_T^{\text{eval}} = \text{Steep}$), whereas policies trained under Steep scheduling perform excellently when evaluated under Fixed or Smooth schedules.

Training	Evaluation	Success / 50 ↑		Metrics at 80% Failure		
Schedule w_T^{train}	Schedule w_T^{eval}	60% Failure	80% Failure	REC ↓	COV ↑	AND
Fixed $(50/50)$	Fixed $(50/50)$	50.0 ± 0.0	$\mathbf{46.6 \pm 3.9}$	0.696 ± 0.048	0.602 ± 0.009	2.578 ± 0.027
	Smooth $(5, 0.7)$	49.2 ± 0.7	42.6 ± 5.0	0.702 ± 0.042	0.613 ± 0.008	2.483 ± 0.026
	Steep $(30, 0.125)$	38.4 ± 3.4	9.8 ± 3.8	0.768 ± 0.070	0.439 ± 0.054	3.828 ± 0.508
Smooth $(5, 0.7)$	Fixed $(50/50)$	50.0 ± 0.0	$\mathbf{47.4 \pm 1.9}$	$\mathbf{0.674 \pm 0.027}$	0.602 ± 0.005	2.580 ± 0.026
	Smooth $(5, 0.7)$	49.6 ± 0.5	43.6 ± 2.8	0.684 ± 0.020	0.616 ± 0.006	2.479 ± 0.058
	Steep $(30, 0.125)$	38.2 ± 5.3	12.0 ± 4.6	0.797 ± 0.037	0.410 ± 0.027	4.057 ± 0.220
Steep $(30, 0.125)$	Fixed $(50/50)$	49.4 ± 0.5	$\mathbf{46.6 \pm 0.5}$	0.679 ± 0.023	0.622 ± 0.004	2.509 ± 0.048
	Smooth $(5, 0.7)$	49.2 ± 0.7	$\mathbf{46.4 \pm 0.5}$	0.706 ± 0.018	$\mathbf{0.624 \pm 0.003}$	$\mathbf{2.474 \pm 0.043}$
	Steep $(30, 0.125)$	41.0 ± 3.3	14.2 ± 4.3	0.796 ± 0.032	0.413 ± 0.032	4.114 ± 0.296

direction field π_θ.

2. **Steep evaluation collapses recovery regardless of how the policy was trained:** Evaluating under $w_T^{\text{eval}} = \text{Steep}$ causes 80%-failure recovery to plummet to 9.8 ± 3.8 (for Fixed-trained policies), 12.0 ± 4.6 (for Smooth-trained policies), and 14.2 ± 4.3 (for Steep-trained policies), while coverage drops by roughly 18 to 21 pp (down to 0.410–0.439) and node clustering spikes (AND rising from ≈ 2.48 to 3.83–4.11). Over the 400-step inference horizon, driving $w_E^{\text{eval}}(t) \to 0$ removes the heuristic link-retention barrier (Proposition 1), causing the learned policy to over-cluster local nodes and stall before bridging distant components.

9.4 Granularity Substitution Counter-Experiment (HSC)

Replacing HAC's node-level global GCN with a subnetwork-level rigid-translation GCN (HSC) causes deterministic recovery collapse at 80% **failure across all five seeds that schedule smoothing cannot rescue.** To verify whether smooth scheduling can compensate for restricted spatial expressiveness, we evaluate HSC across five random seeds under both deployments (Table 8). While smoothing the schedule from $(30, 0.125)$ to $(5, 0.7)$ improves HSC's recovery at 60% failure from 12.2 ± 3.7 to 40.8 ± 1.5 (random) and from 13.0 ± 4.9 to 45.0 ± 2.8 (uniform), all HSC variants collapse to 0.0 ± 0.0–0.4 ± 0.5 out of 50 at 80% failure. When a policy's output space is restricted to rigid component translation, no fusion schedule can supply the node-level degrees of freedom required to bridge severe multi-directional fragmentation.

Table 8: Five-seed granularity substitution counter-experiment with HSC (heuristic branch fused with subnetwork-level rigid translation, Success/50, mean ± std over 5 seeds). Unlike node-level HAC, HSC collapses at 80% failure regardless of schedule smoothness.

	Random Deployment (Success/50 ↑)				Uniform Deployment (Success/50 ↑)			
Variant	20%	40%	60%	80%	20%	40%	60%	80%
HSC-Fixed	49.2 ± 0.7	47.6 ± 1.5	38.0 ± 2.7	0.4 ± 0.5	50.0 ± 0.0	49.6 ± 0.5	46.4 ± 1.4	0.2 ± 0.4
HSC-Steep $(30, 0.125)$	50.0 ± 0.0	47.2 ± 1.5	12.2 ± 3.7	0.0 ± 0.0	50.0 ± 0.0	49.4 ± 0.5	13.0 ± 4.9	0.0 ± 0.0
HSC-Smooth $(5, 0.7)$	50.0 ± 0.0	49.2 ± 0.7	40.8 ± 1.5	0.0 ± 0.0	50.0 ± 0.0	49.8 ± 0.4	45.0 ± 2.8	0.0 ± 0.0

10 Discussion

10.1 Why Simple Hybrids Work and Why Steep Inference Handover Fails

Velocity-level hybrid controllers succeed because the heuristic and learned branches resolve orthogonal failure modes, provided the heuristic link-retention barrier remains active throughout inference. The virtual-force branch guarantees local link retention and inter-UAV spacing within connected components (Proposition 1) but cannot perceive disconnected components beyond R_c. Conversely, the global k-NN GCN perceives the relative geometry of disconnected subnetworks across the entire arena but tends to over-cluster nodes if left unchecked. Our 3×3 decoupled matrix (Table 7) demonstrates that persistent velocity-level coupling during inference is essential: when w_T^{eval} is kept constant (50/50) or transitioned smoothly ($k = 5, s_0 = 0.7$), the heuristic branch continuously regularizes intra-component spacing while the GCN steers clusters into contact, even attaining ±0.5 trial variance when paired with a steep-trained policy.

10.2 Engineering Guidelines for Hybrid Swarm Control

Practitioners designing hybrid recovery controllers should ensure node-level actuation expressiveness first and retain persistent heuristic damping during inference. Comparing HAC and HSC (Table 8) establishes that schedule tuning cannot substitute for output-space degrees of freedom: subnetwork-level rigid translation fails under 80% node loss regardless of gating parameters. Furthermore, Table 7 shows that training and inference schedules need not be identical: pairing an aggressive training schedule with a conservative fixed or smooth evaluation schedule yields the highest coverage (0.624) and lowest variance (±0.5 trials) under extreme failure.

10.3 Limitations and Future Work

Closing the remaining 5.9 to 7.9 pp coverage gap under 80% failure and extending differentiable rollout horizons represent natural directions for future work. While HAC-Fixed and HAC-Smooth retain 88.4% to 91.2% of R2C's reported coverage under 80% node loss without boundary-hull machinery (and up to 91.6% with decoupled Steep-train/Smooth-eval scheduling), introducing lightweight node-level boundary features or deeper spectral graph convolutions could close the remaining margin without reverting to rigid subnetwork partitioning. In addition, extending differentiable rollout horizons beyond $T_{\text{train}} = 40$ steps via truncated backpropagation through time may further align training and evaluation dynamics under long-horizon recovery scenarios.

11 Conclusion

This paper systematically disentangled the roles of decision paradigm, spatial reasoning granularity, and temporal fusion scheduling in self-healing UAV swarm connectivity recovery. Through five-seed benchmarks across random and uniform deployments, we showed that Hybrid Adaptive Convolution (HAC)—which fuses a parameter-free buffered virtual-force heuristic with a single global node-level GCN—outperforms purely heuristic, purely learned, and subnetwork-level rigid baselines while retaining 88.4% to 91.2% of the coverage reported by complex multipartite architectures under 80% node failure without boundary-hull extraction. Furthermore, by executing a five-seed 3×3 decoupled training-versus-evaluation schedule matrix, we falsified the hypothesis that steep adaptive schedules corrupt policy learning during training, proving instead that premature inference-time handover extinguishes the heuristic link-retention barrier during long-horizon rollouts. Combined with granularity-substitution counter-experiments on HSC and our cardinality-invariant connectivity surrogate loss, these findings provide a rigorous, reproducible foundation for designing simple and resilient hybrid swarm controllers.

References

Ameer A. Abbasi and Mohamed Younis. Movement-assisted connectivity restoration in wireless sensor and actor networks. *IEEE Transactions on Parallel and Distributed Systems*, 20(9):1366–1379, 2008.

Kemal Akkaya, Fatih Senel, Aravind Thimmapuram, and Suleyman Uludag. Distributed recovery from network partitioning in movable sensor/actor networks via controlled mobility. *IEEE Transactions on Computers*, 59(2):258–271, 2009.

C. Bradford Barber, David P. Dobkin, and Hannu Huhdanpaa. The quickhull algorithm for convex hulls. *ACM Transactions on Mathematical Software*, 22(4):469–483, 1996.

Ming Chen, Hao Wang, Chih-Yung Chang, and Xin Wei. SIDR: A swarm intelligence-based damage-resilient mechanism for UAV swarm networks. *IEEE Access*, 8:77089–77105, 2020.

Andrea Coletta, Francesco Giorgi, Gaia Maselli, Marco Prata, Daniel Silvestri, Jonathan Ashdown, and Francesco Restuccia. A 2-UAV: Application-aware content and network optimization of edge-assisted UAV systems. In *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*, pages 1–10, 2023.

Stefan Elfwing, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 107:3–11, 2018.

Zhiwei Guo, Yifan Sun, Yue Wang, Chen Fu, and Jing Zhong. An intelligent framework for resilience recovery of FANETs with spatio-temporal aggregation and multi-head attention mechanism. *Computers, Materials & Continua*, 79(2):2375–2398, 2024.

Yogesh K. Joshi. Autonomous recovery from multi-node failure in wireless sensor networks. In *2012 IEEE Global Communications Conference (GLOBECOM)*, pages 652–657, 2012.

Nikolaos Karalias and Andreas Loukas. Erdos goes neural: An unsupervised learning framework for combinatorial optimization on graphs. *Advances in Neural Information Processing Systems*, 33: 6659–6672, 2020.

Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.

Harrshavasan Kurunathan, Hailong Huang, Kai Li, Wei Ni, and Ekram Hossain. Machine learning-aided operations and communications of unmanned aerial vehicles: A contemporary survey. *IEEE Communications Surveys & Tutorials*, 26(1):496–533, 2023.

Jian Li, Ping Yi, Tong Duan, Ze Zhang, Jing Li, Yue Wang, and Jian Yu. Fast connectivity restoration of UAV communication networks based on distributed hybrid MADDPG and APF algorithm. *Ad Hoc Networks*, 171:103785, 2025.

Jun Li, Cheng Zhou, Jia Liu, Min Sheng, Nan Zhao, and Yi Su. Reinforcement learning-based resource allocation for coverage continuity in high dynamic UAV communication networks. *IEEE Transactions on Wireless Communications*, 23(2):848–860, 2023.

Hao Lin and Liang Ding. Multi-hop differential topology based algorithms for resilient network of UAV swarm. *arXiv preprint arXiv:2411.11342v2*, 2025.

Hao Lin, Liang Ding, Song Chen, Fan Yang, and Ling Qian. Multi-hop diffused graph convolution for resilient UAV swarm networks. In *2024 IEEE International Symposium on Broadband Multimedia Systems and Broadcasting (BMSB)*, pages 1–6, 2024.

Fang Liu and Qing Wang. Asymmetric learning for spectral graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 18798–18806, 2025.

Zhangfeng Ma, Bo Ai, Ruisi He, Guangjie Wang, Yong Niu, and Zhangdui Zhong. A wideband non-stationary air-to-air channel model for UAV communications. *IEEE Transactions on Vehicular Technology*, 69(2):1214–1226, 2019.

Zhenqiang Mi and Jenny Y. Yang. Obstacle-avoidance connectivity restoration for mobile sensor systems with local information. In *2015 IEEE International Conference on Communications (ICC)*, pages 6395–6399, 2015.

Zhenqiang Mi, Yang Yang, and Guangming Liu. HERO: A hybrid connectivity restoration framework for mobile multi-agent networks. In *2011 IEEE International Conference on Robotics and Automation*, pages 1702–1707, 2011.

Zhiyu Mou, Feifei Gao, Jian Liu, and Qing Wu. Resilient UAV swarm communications with graph convolutional neural network. *IEEE Journal on Selected Areas in Communications*, 40(1):393–411, 2022.

Dor Mutzari, Yonatan Aumann, and Sarit Kraus. Heterogeneous multi-robot graph coverage with proximity and movement constraints. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 14646–14654, 2025.

Hao Ni, Qiuming Zhu, Bo Hua, Kai Mao, Yuxin Pan, Farman Ali, Weizhi Zhong, and Xiaomin Chen. Path loss and shadowing for UAV-to-ground UWB channels incorporating the effects of built-up areas and airframe. *IEEE Transactions on Intelligent Transportation Systems*, 25(11): 17066–17077, 2024.

Yabin Peng, Chang Liu, Shaoxun Liu, Yuan Liu, and Yue Wu. SmartTRO: Optimizing topology robustness for internet of things via deep reinforcement learning with graph convolutional networks. *Computer Networks*, 218:109385, 2022.

Yabin Peng, Jun Wu, Tong Duan, Yuan Liu, Ze Zhang, and Jie Zhang. Prioritized recovery strategy for robust UAV swarm communication via graph reinforcement learning. *IEEE Internet of Things Journal*, 12(13):23891–23904, 2025.

Yabin Peng, Chenyu Zhou, Hainan Cui, Tong Duan, Haoyang Chen, Fan Zhang, and Shaoxun Liu. Resilient UAV swarm with fast connectivity recovery and extensive coverage. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 917–925, 2026.

Jan Rychlewski. On hooke's law. *Journal of Applied Mathematics and Mechanics*, 48(3):303–314, 1984.

Ran Shi, Xin Yu, Yue Wang, Ye Tian, Zhi Liu, Wei Wu, Xiao-Ping Zhang, and Manuela M. Veloso. Symmetry-informed MARL: A decentralized and cooperative UAV swarm control approach for communication coverage. *IEEE Transactions on Mobile Computing*, 24(9):8039–8056, 2025.

Siddharth Shriwastav. Round-table negotiation for fast restoration of connectivity in partitioned wireless sensor networks. *Ad Hoc Networks*, 77:11–27, 2018.

Mehmet Tosun, Umut Can Cabuk, Emre Haytaoglu, Orhan Dagdeviren, and Yusuf Ozturk. DP-kCR: Distributed proactive k-connectivity recovery algorithm for UAV-based MANETs. *IEEE Transactions on Reliability*, 73(4):1918–1932, 2024.

Jian Wang, Shuo Li, Di Chen, Qiang Fu, Qi Qi, Hao Sun, Bo He, and Jianxin Liao. Flight trajectory control with network-oriented hierarchical reinforcement learning for UAV-assisted time-sensitive IoT. *IEEE Transactions on Intelligent Transportation Systems*, 26(5):6332–6345, 2025a.

Xu Wang, Zhen Zhao, Liang Yi, Zhaolong Ning, Lei Guo, F. Richard Yu, and Song Guo. A survey on security of UAV swarm networks: Attacks and countermeasures. *ACM Computing Surveys*, 57 (3):1–37, 2024a.

Yao Wang, Wei Chen, Tom H. Luan, Zhou Su, Qichao Xu, Ru Li, and Ning Chen. Task offloading for post-disaster rescue in unmanned aerial vehicle networks. *IEEE/ACM Transactions on Networking*, 30(4):1525–1539, 2022.

Yingze Wang, Zhen Feng, Hui Zhang, Yang Gao, Jing Lei, Le Sun, and Ming Song. Angle robustness unmanned aerial vehicle navigation in GNSS-denied scenarios. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 10386–10394, 2024b.

Yue Wang, Xu Wang, Chao Wei, Qi Ren, and Yong Tang. UAV swarm network topology self-healing via graph-based deep reinforcement learning. In *2025 IEEE Wireless Communications and Networking Conference (WCNC)*, pages 1–6, 2025b.

Zhi Wang, Xin Zhao, Jie Zhang, Nan Yang, Peng Wang, Jian Tang, Jun Zhang, and Liang Shi. APF-CPP: An artificial potential field based multi-robot online coverage path planning approach. *IEEE Robotics and Automation Letters*, 9(11):9199–9206, 2024c.

Zhiqiang Ye, Ke Wang, Yu Chen, Xiang Jiang, and Guanghua Song. Multi-UAV navigation for partially observable communication coverage by graph reinforcement learning. *IEEE Transactions on Mobile Computing*, 22(7):4056–4069, 2022.

Dingyi Zeng, Wanlu Liu, Wenjie Chen, Liang Zhou, Miao Zhang, and Hua Qu. Substructure aware graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 11129–11137, 2023.

Lei Zhang, Yi Du, Jing Xu, and Xin Wang. UAV-enabled IoT: Cascading failure model and topology-control-based recovery scheme. *IEEE Internet of Things Journal*, 11(12):22562–22577, 2024.

A Methodological Verification and Reproducibility Log

To ensure full reproducibility and clarify experimental design choices relative to prior literature (Peng et al., 2026), we document all simulation specifications and numerical verifications below:

- **Failure-node sampling:** Failed UAVs at ratios $\{20\%, 40\%, 60\%, 80\%\}$ are sampled uniformly at random without replacement from the initial deployed swarm.
- **Cardinality-invariant connectivity loss verification:** As derived in Eq. (5), omitting the $-\log(n_i n_j)$ normalization term in a log-sum-exp soft-minimum surrogate introduces a severe component-size bias. On a controlled synthetic benchmark featuring two disconnected components separated by an identical true minimum distance (≈ 100 to $108\,\mathrm{m}$) under two cardinality configurations (3×3 nodes vs. 30×30 nodes), the unnormalized log-sum-exp estimator returned 2.85 for the 3×3 pair and -230.58 for the 30×30 pair. In contrast, our cardinality-invariant estimator in Eq. (5) returned 112.71 and 109.54, respectively, accurately tracking the true geometric separation regardless of cluster size.
- **Boundary buffer specification:** The arena boundary repulsion buffer is set to $d_e = 0.15R_c = 18\,\mathrm{m}$, matching the scale of the inter-UAV equilibrium band $d_u - d_l = 0.1R_c = 12\,\mathrm{m}$.
- **Differentiable rollout horizon:** Training uses $T_{\text{train}} = 40$ differentiable rollout steps per epoch for backpropagation-through-time tractability, while evaluation runs for the full horizon $T = 400$ steps.
- **Grid-based coverage evaluation:** Swarm coverage $C_{\text{cov}}(t)$ is evaluated over a deterministic uniform grid at $5\,\mathrm{m}$ spatial resolution across the $1000 \times 1000\,\mathrm{m}^2$ arena.
- **Full-pipeline random seeding:** Every five-seed experiment locks Python, NumPy, CPU PyTorch (`torch.manual_seed`), and CUDA PyTorch (`torch.cuda.manual_seed_all`) generators prior to both network initialization and environment reset. Before locking network weight initialization, an early unseeded run of GAC-Fixed at 20% failure produced an optimistic outlier ($\mathrm{COV} = 0.830, \mathrm{AND} = 11.87$) compared to the true five-seed distribution ($\mathrm{COV} = 0.747 \pm 0.009, \mathrm{AND} = 13.68 \pm 0.28$), underscoring the necessity of strict multi-seed evaluation.

B Full Hyperparameter Table

C Additional Five-Seed Topology Metrics for Uniform Deployment

Tables 3 and 4 in the main text report full Success/50, REC, COV, and Runtime statistics across all failure ratios. For completeness, the five-seed post-recovery average node degree (AND, mean $\pm$ std) trajectories under uniform deployment across failure ratios ($20\% \to 40\% \to 60\% \to 80\%$) are:

- **HAC-Fixed:** $5.723 \pm 0.011 \to 4.347 \pm 0.020 \to 3.298 \pm 0.038 \to 2.302 \pm 0.033$
- **HAC-Smooth** $(5, 0.7)$**:** $5.722 \pm 0.011 \to 4.333 \pm 0.023 \to 3.237 \pm 0.018 \to 2.278 \pm 0.033$
- **GAC-Fixed:** $5.880 \pm 0.040 \to 6.860 \pm 0.180 \to 8.750 \pm 0.350 \to 5.810 \pm 0.420$
- **SAC-Fixed:** $5.724 \pm 0.010 \to 4.379 \pm 0.035 \to 3.205 \pm 0.000 \to \mathrm{n/a}$
- **Learned-only:** $6.050 \pm 0.300 \to 6.158 \pm 0.248 \to 7.493 \pm 0.916 \to 4.730 \pm 0.549$
- **Heuristic-only:** $5.724 \pm 0.011 \to 4.358 \pm 0.020 \to 2.870 \pm 0.256 \to \mathrm{n/a}$

Table 9: Hyperparameters shared across all evaluated architectures and five-seed training runs.

Parameter	Value
Graph convolution layers L	4
Hidden feature dimension	128
Global k-NN degree k	8
Optimizer	Adam
Learning rate	1×10^{-3}
Gradient norm clipping threshold	1.0
Training epochs	300
Training rollout length T_{train}	40 steps
Evaluation horizon T	400 steps
Independent random seeds per configuration	5 seeds $\times$ 50 trials
Movement cost weight λ	0.01
Heuristic decay rates α, β, δ	0.01
Equilibrium buffer zone $[d_l, d_u]$	$[0.8R_c, 0.9R_c]$
Communication and coverage radii $R_c = R_{\text{cov}}$	120 m
Maximum UAV speed $\hat{v}_{\max}$	1 m/s
Mission area $\mathcal{A}$	$1000 \times 1000\,\text{m}^2$
Random deployment initial UAV count	200
Uniform deployment initial UAV count	196 (14×14 grid)

Consistent with the random-deployment analysis in Figure 2, both HAC-Fixed and HAC-Smooth maintain a low, monotonically decreasing node degree across all failure regimes under uniform deployment, whereas GAC-Fixed and Learned-only exhibit pronounced mid-regime clustering (8.750 and 7.493 at 60% failure).

D Code and Reproducibility Artifacts

The companion repository contains the complete simulation environment (`code/sim/`), heuristic virtual-force and global GCN model definitions (`code/models/`), differentiable rollout training and evaluation scripts (`code/scripts/`), shell scripts for multi-seed execution (`run_full_multiseed_study.sh`), and aggregated five-seed CSV logs (`code/results/`) reproducing every table and figure in this manuscript.